

MiniOneRec 后续研究 进度日志

工作记录文档

最后更新：2026-04-10

1 文档用途

这份文档是当前MiniOneRec 后续研究的持续更新日志，不是正式论文草稿。它的主要目标有四个：

- 持续同步当前研究主线与最新实验结论；
- 记录每一轮实验实际做了什么、改了什么；
- 区分已经比较稳定的事实和仍然开放的想法；
- 为后续正式写论文保留一条清晰、可回溯的叙事链条。

2 当前研究状态

2.1 当前方向

当前研究方向可以概括为：

用图结构作为协同信息的载体，并以层级感知的方式，把graph-structured collaborative information 融合进MiniOneRec 的语义SID 构建过程。

这里的重点不是比较哪一种graph encoder 更强，而是回答三个核心问题：

1. 什么样的图适合承载协同信息？
2. 图结构协同信息应该怎样在不同SID level 上体现层级感知？
3. 图协同信息应该如何进入MiniOneRec 的纯语义SID，而不破坏语义backbone 的稳定性？

2.2 为什么转向这个方向

当前方向主要来自以下几条逐步稳定下来的观察：

- SID collision 存在，但看起来不是主矛盾；
- 当前SID 主要由文本语义驱动，缺少直接协同约束；
- 大量错误表现为“prefix 对了，但leaf 错了”，说明local leaf ambiguity 更像核心瓶颈；
- 早期naive front-end collaborative fusion 不但没有稳定提升，反而可能破坏量化结构。

3 当前较稳定的事实

到目前为止，以下结论已经足够稳定，可以继续指导方法设计：

1. 局部歧义是真问题。模型在进入正确语义区域后，最后一层leaf 仍然经常选错。
2. 图协同信号是有用的。即使还没有改训练版SID，graph-structured collaborative signal 已经在top-1 SID error 上提供额外判别力。
3. 中尺度图视图很重要。最有价值的协同视图既不是纯global，也不是纯last-step local。
4. 轻量净化是必要的。raw collaborative graph 往往弱于purified graph。
5. 论文启发的频域中尺度图比第一版heuristic middle-view 更强。
6. 当前仓库默认的sid-train 超参数与上游MiniOneRec 不一致。真正能复现低collision SID 的，是upstream-style 的RQ-VAE 训练制度，而不是当前默认的epochs=10，batch_size=256。
7. 不能只看sid-train collision。最终真正有意义的是sid-generate 后的index.json collision；train-stage 排名和final index 排名可能并不一致。
8. Hierarchy-aware graph regularization 已经在final SID 上给出正信号。在upstream-aligned regime 下，Industrial 上hierarchy_reg 的final generated SID collision 低于强语义baseline。
9. 这种tokenizer 改进已经部分传到SFT。当前hierarchy 版本在Industrial 上提升了NDCG@3 和HR@3，但还没有形成全面的@10+ 优势。
10. 下游收益主要集中在crowded local ambiguity。evaluate 诊断表明，hierarchy 主要修复的是same- l_2 与高fanout 目标上的局部错误，而不是一个均匀的全局收益。

4 被削弱或阶段性放下的方向

4.1 Naive 前端协同融合

之前的实验说明，粗糙的前端语义加协同融合会破坏SID 结构并伤害量化稳定性。这并不意味着所有前端协同融合都是错的，只是否掉了那一版过于粗糙的融合方式。

4.2 把后验局部修正当成最终论文主线

局部后验修正仍然是很有用的诊断工具和原型验证工具，但单独作为最终论文主方法太弱。更有价值的主线，还是把图结构协同信息以层级感知方式写回SID 构建本身。

5 实验时间线

5.1 阶段0：问题收敛

这一阶段的主要贡献还是概念层面的。项目从一个较宽泛的“往SID 里加协同信息”方向，逐渐收敛到更明确的表述：

图结构承载的协同信息，应该以层级感知且保结构的方式进入SID 构建。

这一阶段同时也固定了三个核心问题，并明确了图在当前研究中的身份：

图不是主要比较对象，而是collaborative information 的结构化载体。

5.2 阶段1：初始Graph-Bank Probe (2026-04-09)

做了什么

我们新增了一套独立probe，不修改MiniOneRec 原有训练或评测主链，只在已有top-1 SID 错误样本上测试不同graph view 的判别能力。测试的视图包括：

- coarse collaborative graph;
- local transition graph;
- heuristic middle-resolution views:
 - diffusion residual;
 - band-pass proxy;
 - community-aware view。

其中适用时分别比较raw 与purified 版本。

主要结论

第一轮里最有价值的middle-resolution views 来自purified heuristic views:

- Industrial: 最好的middle view 是mid_band_pass_purified, same_l2 = 0.2747
- Office: 最好的middle view 是mid_diffusion_purified, same_l2 = 0.2381

这一轮的意义是明确了：

middle-resolution collaborative structure 值得被当成一个一等设计对象，而不是顺手派生出来的附属信号。

说明了什么

这轮结果支持下面这个解释:

对深层leaf ambiguity 最有用的协同信息, 既不是纯global, 也不是纯local。中尺度视图是必要的。

5.3 阶段2: 论文模块移植Probe (2026-04-09)

做了什么

第二轮probe 依然保持为独立实验模块, 但移植了两类更接近相关工作的思想:

- PRISM 风格的semantic-anchor purification
- GSPRec / FaGSP 风格的频域middle-resolution graph views

这一轮仍然没有改SID 训练本体, 而是在同样的top-1 error probe 框架下测试更强的graph carrier。

主要结果

视图类型	Industrial 好same_12	最 Office 好same_12	最 说明
第一版heuristic middle-view	0.2747	0.2381	上一轮最强purified proxy
移植后的spectral middle-view	0.5401	0.7619	都来自fagsp_mid_base

更具体地说:

- Industrial:
 - 上一轮最强中尺度视图: mid_band_pass_purified = 0.2747
 - 当前最强移植视图: fagsp_mid_base = 0.5401
 - 第二强移植视图: gsprec_mid_prism = 0.5247
- Office:
 - 上一轮最强中尺度视图: mid_diffusion_purified = 0.2381
 - 当前最强移植视图: fagsp_mid_base = 0.7619
 - 第二强移植视图: gsprec_mid_prism = 0.6429

说明了什么

这一轮是目前为止最强的一次正信号：

当前的`G_mid` 很可能应该被设计为一个真正的graph-spectral middle-resolution collaborative view，而不是只依赖heuristic diffusion / band-pass proxy。

同时，这轮结果也说明：

semantic-anchor purification 本身是有帮助的，但真正的大幅增益主要来自更强的中尺度结构表达，而不只是anchor-only denoising。

5.4 阶段3：训练脚手架校准（过渡阶段）

在进入正式训练验证之前，我们做过几轮训练脚手架和配置校准，用来确认两件事：

- 独立实验分支可以在不破坏baseline 主链的前提下稳定运行；
- 真正有意义的方法比较必须建立在与MiniOneRec 上游一致的训练制度上。

这一阶段的详细过程不再作为当前主线证据保留。对当前研究真正有价值的结论只有一条：

在做graph-aware SID 对比之前，必须先解决baseline provenance 与训练制度对齐问题。

5.5 阶段4：SID provenance 追踪与上游风格复现（2026-04-09）

做了什么

为了确认当前正式Industrial SID 的真实来源，我们专门做了两件事：

- 对比本地迁移版与上游MiniOneRec 官方仓库中的`rqvae.py`、`trainer.py`、`generate_indices.py` 核心逻辑；
- 在本地主线代码上，按上游官方训练超参数重新跑一遍Industrial 的RQ-VAE -> `sid-generate`。

主要结果

对比上游后发现：

- 核心`sid-train` / `sid-generate` 代码逻辑没有被迁移改坏；
- 真正的关键差异在于训练制度：
 - 上游官方：`epochs = 10000`, `batch_size = 20480`
 - 当前本地默认：`epochs = 10`, `batch_size = 256`

随后，按上游风格重跑Industrial:

- `sid-train` 最佳collision 降到 ≈ 0.0993 ;
- 继续跑`sid-generate` 后，最终`index.json` collision 变为 ≈ 0.0043407488 ;
- 这个值与仓库中当前正式Industrial index 的collision 完全一致。

说明了什么

这一轮非常关键，因为它修正了我们对基线系统的理解:

当前正式Industrial SID 是可以被当前主线代码复现出来的，问题不在于迁移时改坏了主逻辑，而主要在于本地默认训练超参数偏离了上游MiniOneRec。

这也意味着，后续所有graph-aware 实验如果还想保持说服力，就必须在`upstream-aligned` 训练制度下进行。

5.6 阶段6: Upstream-aligned MGR-SID v1 训练与生成 (2026-04-09 / 2026-04-10)

做了什么

在确认基线provenance 后，我们把MGR-SID v1 的训练实验也切换到了与上游对齐的训练制度:

- `epochs = 10000`
- `batch_size = 20480`
- Industrial only
- 三组对照:
 - baseline
 - uniform graph regularization
 - hierarchy-aware graph regularization

训练结束后，我们没有停留在`sid-train` 指标，而是对每个best checkpoint 单独跑了实验版generate, 比较最终`index.json` collision。

主要结果

Generate-stage final collision

- baseline: ≈ 0.0035268584
- uniform_reg: ≈ 0.0059685296

- `hierarchy_reg`: ≈ 0.0032555616

其中最关键的是：

- `hierarchy_reg` 成为三者里最终最好的SID；
- `baseline` 也比当前仓库里原有正式Industrial index 的 ≈ 0.0043407488 更低；
- 说明这轮upstream-aligned 训练本身已经把语义baseline 推到了更健康的regime。

说明了什么

这是当前研究里最重要的新转折之一：

当前最重要的判断标准应该是final generated SID，而不是单独盯住raw `sid-train collision`；在这个更合理的判据下，当前这版hierarchy-aware graph regularization 已经给出了第一轮清晰正信号。

更具体地说，这一轮至少说明了三件事：

- `hierarchy-aware` 方向不是空想，它已经在最终Industrial SID 上给出正结果；
- 后续评估应该优先围绕**final generated SID** 展开，而不是再把全部判断压在train-stage collision 上；
- 当前实验主线已经具备继续推进的依据，不再只是probe 级别的希望信号。

5.7 阶段7：实验数据转换与第一轮SFT + Evaluate (2026-04-10)

做了什么

在确认final generated SID 上已经有正信号之后，我们先没有进入RL，而是做了最小下游验证。具体分成两步：

- 基于两套Industrial tokenizer 结果，建立隔离的`data_experiment` 目录，保持与现有`data/Amazon` 相同的结构，只替换：
 - `history_item_sid`
 - `item_sid`
- 然后对两套数据分别跑平行的SFT -> evaluate:
 - `mgr_upstream_baseline`
 - `mgr_upstream_hierarchy`

主要结果

两条链最终都完整跑通。Industrial 上的关键指标如下：

- baseline:
 - $\text{NDCG@1} = 0.06309$
 - $\text{NDCG@3} = 0.07772$
 - $\text{NDCG@10} = 0.09430$
 - $\text{HR@1} = 0.06309$
 - $\text{HR@3} = 0.08824$
 - $\text{HR@10} = 0.13435$
- hierarchy:
 - $\text{NDCG@1} = 0.06265$
 - $\text{NDCG@3} = 0.08025$
 - $\text{NDCG@10} = 0.09360$
 - $\text{HR@1} = 0.06265$
 - $\text{HR@3} = 0.09287$
 - $\text{HR@10} = 0.13038$

最直接的delta 是：

- $\text{NDCG@3} = +0.00253$
- $\text{HR@3} = +0.00463$
- NDCG@5/HR@5 也有轻微提升
- 但 NDCG@1/HR@1 和 @10+ 仍然弱于baseline

说明了什么

这一轮的意义不是“hierarchy 已经全面赢了”，而是更细的：

tokenizer 侧的结构收益已经可以部分传到下游，但目前主要体现为短程排序增益，而不是全面的end-to-end 优势。

这说明两件事：

- 当前方向不是只在tokenizer 原地自嗨，它确实会影响下游行为；
- 但当前tokenizer-to-SFT 的传递还不够理想，仍然存在一部分收益在下游被抵消。

5.8 阶段8: SFT 历史对比与Evaluate 错误分布分析 (2026-04-10)

做了什么

在拿到第一轮SFT 结果后, 我们又补了两类分析:

- 与项目内历史Industrial SFT 记录做主指标对比;
- 对baseline 与hierarchy 的最终evaluate 输出做SID-aware 错误分布诊断。

主要结果

与历史CSV 的对比 结果显示, 当前两条新实验都还没有超过历史最强Industrial SFT 设定:

- 历史最强: title_history2sid_off + desc_align_p05
- 当前hierarchy: 只在NDCG@3 上略高于“最接近当前设定”的历史baseline, 并在HR@3 上持平
- 但在@10+ 指标上, 当前baseline 与hierarchy 都还偏弱

Evaluate 错误分布 这轮分析给出了目前最重要的机制性证据。核心数字包括:

- top-1 error same- l_1 rate: 0.1815 $\rightarrow$ 0.1236
- top-1 error same- l_2 rate: 0.0525 $\rightarrow$ 0.0370
- crowded targets (target l_2 fanout ≥ 4) 上的top-1 hit: 0.0475 $\rightarrow$ 0.0521
- baseline same- l_2 errors 的恢复率: 0.0538

逐样本对比还显示:

- hierarchy 修复了63 个baseline top-1 miss;
- 但也打坏了65 个baseline 原本正确的top-1 hit;
- fixed cases 的平均target l_2 fanout 高达7.32, 说明它们大多是更拥挤、更难的局部歧义样本;
- broken cases 则往往是baseline 本来就已经进入正确局部子树的样本。

说明了什么

这一轮把“为什么hierarchy 只在短程指标上更好”解释清楚了:

hierarchy-aware SID 的确在修我们关心的crowded local ambiguity, 但它当前也会打掉一部分baseline 原本已经正确的局部routing。

也就是说, 当前瓶颈已经不再是“tokenizer 方向对不对”, 而更像是:

如何让tokenizer 侧的局部歧义收益, 在SFT 阶段被更完整地保留下来, 而不是同时损伤coarse/local retention。

6 当前证据的整体解释

把目前的证据串起来，可以得到一个更清楚的图景：

1. 当前semantic SID 足以提供较强的coarse routing;
2. 剩余的主要困难集中在local leaf-level discrimination;
3. graph-structured collaborative signal 对这个瓶颈是有帮助的;
4. 当前最强候选是**spectral middle-resolution collaborative graph view**;
5. 当前仓库默认**sid-train** 配置不能代表正式MiniOneRec baseline，所有方法对比必须切到upstream-aligned regime;
6. 在upstream-aligned regime 下，**hierarchy-aware** 已经在final generated SID 上给出了第一轮正信号;
7. 把新的SID 推进到SFT 之后，收益主要先体现在短程排序和crowded local ambiguity 样本上;
8. 当前没有形成全面下游增益的主要原因，不像是tokenizer 方向错误，而更像是tokenizer-to-SFT transfer 仍然会牺牲一部分baseline 已经正确的局部routing;
9. 这说明图信号本身有用，而且图信号如何同时与**sid-generate** 的collision resolution 以及SFT 阶段的排序行为交互，是方法设计里非常关键的两道接口。

7 当前最佳工作假设

当前最可信的工作假设是：

MiniOneRec 应该继续保留semantic SID 作为backbone，同时加入hierarchy-aware graph regularization；其中middle-resolution spectral collaborative graph 是降低local leaf ambiguity 的关键额外信号，而真正的有效性需要分两层判断：先看**sid-generate** 后的final SID，再看这些结构收益能否在SFT 中被尽量无损地保留下来。

目前最强的G_mid v1 候选是：

- 主候选: **fagsp_mid_base**
- 备选: **gsprec_mid_prism**

当前最合理的控制组地位也更清楚了：

- **hierarchy_reg**: 当前主方法候选
- **baseline**: 强语义基线

8 当前开放问题

即使有了新一轮probe, 下面这些问题仍然没有完全解决:

1. 当前final-SID 改善是否能在第二个seed 上保持?
2. baseline 与hierarchy_reg 的最终index 差异, 是否真的主要集中在same₁₂ / local ambiguity bucket?
3. 当前hierarchy_reg 的收益究竟来自更好的leaf disambiguation, 还是更好的collision repair compatibility?
4. semantic anchoring 在训练时应该如何使用, 才能避免把graph signal 限制得过死?
5. graph regularization 最佳作用位置到底是encoder latent、quantized residual, 还是cumulative quantized representation?
6. 当前SFT 中被打坏的那批baseline-correct local routing, 究竟来自SID 分布变化、训练目标不匹配, 还是decoding/beam 行为变化?
7. 如何保住hierarchy 在crowded same- l_2 case 上的收益, 同时减少对baseline 已正确局部邻域的破坏?
8. 什么时候适合把published baselines 拉进主表, 避免在机制还没站稳前过早铺开大规模复现?

9 下一步行动

当前最合理的next actions 是:

1. 保留当前graph bank, 不再频繁改carrier:
 - G_{coarse}: purified coarse collaborative graph;
 - G_{mid}: fagsp_{mid}_base;
 - G_{local}: purified transition graph。
2. 优先围绕final generated SID 做分析, 而不是再把注意力只放在sid-train collision 上;
3. 下一轮最优先比较:
 - baseline;
 - hierarchy-aware graph regularization。
4. 下一阶段不要先扩更多tokenizer idea, 而应优先改tokenizer-to-SFT transfer, 目标是:
 - 保住hierarchy 对crowded local ambiguity 的修复;
 - 尽量不打坏baseline 已经正确的coarse/local routing。
5. 在此基础上, 再补:
 - second seed 的final SID 验证;

- SFT 下的重复运行或小范围机制改写;
 - 必要时再进入RL。
6. 如果transfer 问题开始被解决, 再考虑做:
- Office 上的对应实验;
 - published baselines 主表准备。

10 当前关键材料位置

当前阶段最关键的材料包括:

- 任务对齐文档: `idea-discovery/.../CURRENT_TASK_ALIGNMENT.md`
- 第一轮probe 记录: `idea-discovery/.../13_initial_probe_run_2026-04-09.md`
- 论文模块移植probe 记录: `idea-discovery/.../14_paper_transplant_probe_run_2026-04-09.md`
- 上游风格SID 复现记录: `research-progress-log/experiment_launches/2026-04-09_upstream_style_rqvae_result.md`
- upstream-aligned MGR-SID v1 总记录: `research-progress-log/experiment_launches/2026-04-09_mgr_sid_v1_upstream/README.md`
- upstream-aligned MGR-SID v1 结果记录: `research-progress-log/experiment_launches/2026-04-09_mgr_sid_v1_upstream/RESULTS.md`
- SFT 数据转换记录: `research-progress-log/experiment_launches/2026-04-10_mgr_sid_data_experiment_convert/README.md`
- Industrial SFT + evaluate 结果: `research-progress-log/experiment_launches/2026-04-10_mgr_sid_sft_eval_industrial/RESULTS.md`
- 与历史结果对比: `research-progress-log/experiment_launches/2026-04-10_mgr_sid_sft_eval_industrial/COMPARE_TO_HISTORY.md`
- evaluate 错误分布分析: `research-progress-log/experiment_launches/2026-04-10_mgr_sid_sft_eval_industrial/EVAL_ERROR_DISTRIBUTION.md`
- 第一轮probe 脚本: `scripts/experiment_mgr_sid_graph_bank_probe.py`
- 模块移植probe 脚本: `scripts/experiment_mgr_sid_paper_transplant_probe.py`
- 实验版训练脚本: `scripts/experiment_mgr_sid_v1_train.py`
- 实验版生成脚本: `scripts/experiment_mgr_sid_v1_generate.py`
- 实验版数据转换脚本: `scripts/experiment_mgr_sid_prepare_data.py`
- 实验版局部歧义分析脚本: `scripts/experiment_mgr_sid_v1_local_ambiguity_analysis.py`